

UNIVERSITY OF SOUTHERN DENMARK

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

SPDM801: MASTER THESIS IN COMPUTER SCIENCE

Formalising π LL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

1st June 2026



Abstract

This thesis presents a mechanised formalisation of π LL, a process calculus relating linear logic and session-typed communication, in Lean 4. The formalisation covers the static structure of the full calculus, including session types, processes, environments, hyperenvironments, typing judgements, and substitution principles. To represent binding, the development retains explicit free identifiers while encoding bound occurrences by indices: channel binders are handled through a locally nameless representation with co-finite premises, while polymorphic type binders are treated through type-depth indexing, shifting, and substitution. The operational semantics and session-fidelity development are mechanised for the multiplicative fragment, π MLL, using a derivation-level transition system together with projected process-level and environment-level relations. The completed metatheory establishes the projection of typed transitions to process and resource transitions and develops the reverse lifting argument for all multiplicative cases except the restriction case. The remaining *res* case is isolated to the reconstruction of endpoint-bearing hyperenvironment components after a transition under restriction. The development demonstrates that mechanising the paper theory in this way can relocate implicit reasoning about bound names, freshness, and unordered linear resources into explicit structural invariants. It thereby contributes a mechanised account of π LL and identifies the remaining infrastructure required to complete session fidelity for π MLL.

Resumé

Denne afhandling præsenterer en mekaniseret formalisering af π LL, en proceskalkule, der forbinder lineær logik med sessionstypet kommunikation, i Lean 4. Formaliseringen dækker den statiske struktur af den fulde kalkule, herunder sessionstyper, processer, miljøer, hypermiljøer, *typing judgements* og substitutionsprincipper. Til repræsentation af binding fastholder udviklingen eksplicitte frie navne, mens bundne forekomster repræsenteres ved indeks: kanalbindere håndteres gennem en *locally nameless* repræsentation med co-finite-præmisser, mens polymorfe typebindere behandles gennem typedybdeindeksering, forskydning og substitution. Den operationelle semantik og udviklingen af *session fidelity* er mekaniseret for det multiplikative fragment, π MLL, ved hjælp af et transitionssystem over *typing derivations* samt projicerede transitionsrelationer for processer og hypermiljøer. Den gennemførte metateori etablerer projektionen af typede transitioner inkluderende til proces- og ressourcetransitioner og udvikler det omvendte løfteargument for alle multiplikative tilfælde undtagen restriktionstilfældet. Det resterende *res*-tilfælde er isoleret til rekonstruktionen af de hypermiljøkomponenter, der bærer de restrikerede endepunkter, efter en transition under restriktion. Udviklingen viser, at en mekanisering af papirteorien på denne måde kan flytte implicit ræsonnement om bundne navne, friskhed og uordnede lineære ressourcer til eksplicitte strukturelle invarianter. Afhandlingen bidrager dermed med en mekaniseret redegørelse for π LL og identificerer den resterende infrastruktur, der kræves for at fuldføre sessionsfidelitet for π MLL.

CONTENTS

1	Background	3
1.1	Classical Linear Logic, CLL	3
1.2	Classical Linear Logic, CLL	3
1.3	The π -Calculus: Processes and Communication	4
1.4	Types and Propositions-as-Sessions Correspondence	5
1.5	Environments, Hyperenvironments and Judgements	5
1.6	Operational Semantics	7
1.7	Process Semantics and Erasure	9
1.8	Semantics of Typing Environments	11
1.9	Managing Bound Variables	12
	References	13

1 Background

This section begins by introducing the theoretical foundations on which the formalisation is based, then following Montesi and Peressotti [2021] in the present of the full π LL calculus, including processes, types, typing, environments / hyperenvironments, and judgements, then restrict attention to the multiplicative fragment π MLL when discussing operational semantics.

Following Montesi and Peressotti [2021], we use *blue* to highlight types and *red* for process terms throughout this thesis.

1.1 Classical Linear Logic, CLL

Linear logic [Girard 1987] is a refinement of classical and intuitionistic logic. In contrast to these systems, where formulas can be freely duplicated or discarded, linear logic restricts structural rules such as weakening and contraction, and instead treats formulas as consumable resources. This yields a resource-sensitive interpretation of logic, where each assumption must be used in a controlled manner [Di Cosmo and Miller 2023]. In this thesis, we work in the classical presentation of linear logic, where sequents are symmetric and duality replaces the distinction between assumptions and conclusions.

1.2 Classical Linear Logic, CLL

Linear logic [Girard 1987] is a refinement of classical and intuitionistic logic. In contrast to these systems, where assumptions can be duplicated or discarded freely, linear logic controls the use of structural rules such as weakening and contraction, thereby giving formulas a resource-sensitive interpretation [Di Cosmo and Miller 2023].

The multiplicative fragment forms the core of the system and includes the tensor connective $A \otimes B$ and its dual $A \wp B$, together with the corresponding units 1 and $\perp$. These connectives capture composition and interaction of resources.

Intuitively, $A \otimes B$ describes the composition of two independent resources, while $A \wp B$ captures their dual form of interaction. Under a process interpretation, $\otimes$ can be read as producing a value of type A and continuing as B , whereas $A \wp B$ corresponds to receiving a value of type A and proceeding as B . The units 1 and $\perp$ represent the absence of further output and input, respectively. A simplified presentation of these rules is given below:

$$\frac{}{\vdash 1} \quad \frac{}{\vdash \Gamma, \perp} \quad \frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B} \otimes \quad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \wp B} \wp$$

Here rule $\otimes$ presents a simplified form of the tensor rule from classical linear logic. In general, the tensor connective $A \otimes B$ combines two independent derivations. It requires a proof of A and a proof of B , each obtained from disjoint contexts, and produces a proof of the composite resource $A \otimes B$. Intuitively, this corresponds to combining two independent resources into a single composite resource, and can be read as producing a value of type A and then continuing as B .

The unit rule *rule 1* introduces the unit 1, which represents the absence of further obligations. It can be understood as an *empty output*, corresponding to a computation that performs no further interaction, and serves as the neutral element of tensor.

Dually, rules $\perp$ and $\wp$ describe the behavior of the connective $A \wp B$ and its unit $\perp$. The $\wp$ rule combines resources within a single context, reflecting a form of interaction where both components are made available to the surrounding context. From an operational perspective, this corresponds to interacting with an input of type A while continuing as B . The unit $\perp$ represents the absence of further input, acting as an *empty input* and serving as the dual neutral element.

For example, two independent instances of the unit 1 can be introduced by the $\mathbf{1}$ rule and combined via the $\otimes$ rule to derive $1 \otimes 1$. In this case the surrounding context Γ and Δ are empty:

$$\frac{\frac{}{\vdash 1} 1 \quad \frac{}{\vdash 1} 1}{\vdash 1 \otimes 1} \otimes$$

Here each leaf introduces one resource independently, and the tensor rule combines them into a single compound resource $1 \otimes 1$. After this step, the individual resources are no longer available separately, reflecting the resource-sensitive nature of the system.

This resource-sensitive view of logic provides a foundation for a use in computer science, particularly in the study of concurrency. In the following sections, this perspective is used to motivate the process interpretation underlying π LL.

1.3 The π -Calculus: Processes and Communication

The π -calculus [Milner et al. 1992] is a process calculus for modeling concurrent computation with dynamic communication topology, where channel names themselves can be transmitted, allowing the communication network to evolve during execution. We follow the presentation of Montesi and Peressotti [2021], which adopts Wadler’s convention of using square brackets ‘[-]’ for output and round parentheses ‘(-)’ for input.

Programs in π LL are processes $(P, Q, R, \dots)$, which communicate over names $(x, y, z, \dots)$ representing session endpoints. Sessions are binary, a discipline enforced by the typing system introduced in Section 1.4. Following Montesi and Peressotti [2021], we assume an infinite set of names with decidable equality. The process terms of π LL are defined by the following grammar, reproduced from Montesi and Peressotti [2021] with minor adjustments:

$P, Q ::=$	$x[y].P$	<i>output y on x and continue as P</i>
	$ \quad x(y).P$	<i>input y on x and continue as P</i>
	$ \quad x[].P$	<i>output (empty message) on x and continue as P</i>
	$ \quad x().P$	<i>input (empty message) on x and continue as P</i>
	$ \quad \nu xy P$	<i>name restriction (cut)</i>
	$ \quad P \mid Q$	<i>parallel composition</i>
	$ \quad x[L].P$	<i>select left on x and continue as P</i>
	$ \quad x[R].P$	<i>select right on x and continue as P</i>
	$ \quad x.\text{case}\{L:P, R:Q\}$	<i>offer a binary choice between P or Q on x</i>
	$ \quad x[A].P$	<i>output type A on x and continue as P</i>
	$ \quad x(X).P$	<i>input a type as X on x and continue as P</i>
	$ \quad !x\langle z_1, \dots, z_n \rangle. \{P\}$	<i>server (replicable process) on x depending on servers on $z_1 \dots z_n$</i>
	$ \quad x[\text{USE}].P$	<i>consume a server on x and continue as P</i>
	$ \quad x[\text{DUP}](x').P$	<i>duplicate server x as x' in P</i>
	$ \quad x[\text{DISP}].P$	<i>dispose of a server on x</i>
	$ \quad x \leftrightarrow y$	<i>link x and y</i>
	$ \quad \mathbf{0}$	<i>terminated process</i>

Term $x[y].P$ allocates a fresh name y , outputs it over x , and proceeds as P . Dually, $x(y).P$ takes input over x , binding it to y in P . Terms $x[].P$ and $x().P$ model empty output and input. Restriction $\nu xy P$ connects the two endpoints x and y of P to form a session, where the order of x and y is

irrelevant and $\nu xy P \equiv \nu yx P$. Parallel composition $P \mid Q$ runs P and Q concurrently, and 0 is the terminated process, acting as the unit of parallel composition.

Example 1.1. Consider the process $P \triangleq \nu xz (x[] \cdot 0 \mid z().y[] \cdot 0)$. This process creates a private session between the restricted endpoints x and z . The left process, $x[] \cdot 0$, signals termination on x before terminating, while the right process, $z().y[] \cdot 0$, waits for a termination signal on z before proceeding by sending a termination signal on y and then terminating.

This calculus provides the operational foundation for π LL. In the following section, we introduce the type system that governs how names are used, ensuring that each session endpoint is used exactly according to its protocol.

1.4 Types and Propositions-as-Sessions Correspondence

In π LL, session types $(A, B, C, \dots)$ are linear logic propositions [Caires and Pfenning 2010; Montesi and Peressotti 2021; Wadler 2014]. Each type describes the communication protocol of a channel endpoint, and duality ensures that the two endpoints of every session implement complementary protocols.

The account of session types used in this thesis follows the presentation of Montesi and Peressotti [2021], which builds on the propositions-as-sessions correspondence developed by Carbone et al. [2016] and Wadler [2014]. In this correspondence, each connective of linear logic is interpreted as a communication protocol, where $A \otimes B$ describes sending a value of type A and continuing as B , while its dual $A \wp B$ describes receiving. The units 1 and $\perp$, again, represent the absence of out and input, respectively. The additive connectives $A \oplus B$ and $A \& B$ capture choice in the sense of selection and offering of alternatives, while the exponentials $!A$ and $?A$ govern replication. The $\forall X.A$ and $\exists X.A$ allow polymorphic session behavior. The type grammar of π LL is defined by Montesi and Peressotti [2021] as follows:

$A, B ::=$	$A \otimes B$	send A , proceed as B	$A \wp B$	receive A , proceed as B
	1	empty output, unit for $\otimes$	$\perp$	empty input, unit for $\wp$
	$A \& B$	offer A or B	$A \oplus B$	select A or B
	$\exists X.A$	existential, type output	$\forall X.A$	universal, type input
	$?A$	client request	$!A$	server accept
	X	type variable	$X^\perp$	dual of type variable

Duality is defined structurally on session types by exchanging each connective with its dual and forms an involution, specifically $(A^\perp)^\perp = A$. Consequently, the endpoints of a well-typed session are always assigned dual types, ensuring they are compatibility:

$$\begin{aligned}
 (A \otimes B)^\perp &= A^\perp \wp B^\perp & 1^\perp &= \perp & (A \oplus B)^\perp &= A^\perp \& B^\perp \\
 (!A)^\perp &= ?A^\perp & (\exists X.A)^\perp &= \forall X.A^\perp & (X)^\perp &= X^\perp
 \end{aligned}$$

1.5 Environments, Hyperenvironments and Judgements

Typing environments in π LL are organised into two layers. Environments track how individual names are typed, while hyperenvironments track which names are used independently in parallel.

Following the presentation in Montesi and Peressotti [2021], *environments* $(\Gamma, \Delta, \Xi, \dots)$ are defined as lists allowing exchange, meaning order is irrelevant. They can be written as $\Gamma = x_1 : A_1, \dots, x_n : A_n$, where each x_i is associated to its respective A_i , for $1 \leq i \leq n$. Environments can be merged as long as they do not share any names for example assume Γ and Δ do not share any names, then Γ, Δ is

the environment containing exactly the entries of Γ and Δ regardless of ordering. Environments act as a partial commutative monoid, using “,” as sum and the empty environment $\bullet$ as unit:

$$\Gamma, \bullet = \Gamma \quad \Gamma, \Delta = \Delta, \Gamma \quad (\Gamma, \Delta), \Xi = \Gamma, (\Delta, \Xi)$$

Environments dictate how names are used, but it does not distinguish whether names are used independently or in parallel. That role is handled by *hyperenvironments* $(\mathcal{G}, \mathcal{H}, \mathcal{I}, \dots)$, which are collections of environments joined using the parallel operator ‘|’. They can be written as follows $\mathcal{G} = \Gamma_1, \dots, \Gamma_n$, where $\Gamma_1, \dots, \Gamma_n$ is a collection of environments. Given $\mathcal{G}$ and $\mathcal{H}$ having no names in common then $\mathcal{G} | \mathcal{H}$ is the hyperenvironment containing exactly the environments of $\mathcal{G}$ and $\mathcal{H}$, ignoring order. Like environments, all names in a hyperenvironment are unique, they can be combined as long as they do not share any names, and they act as a partial commutative monoid using ‘|’ as sum and $\emptyset$ as unit, where $\emptyset$ is the hyperenvironment containing no environments:

$$\mathcal{G} | \emptyset = \mathcal{G} \quad \mathcal{G} | \mathcal{H} = \mathcal{H} | \mathcal{G} \quad (\mathcal{G} | \mathcal{H}) | \mathcal{I} = \mathcal{G} | (\mathcal{H} | \mathcal{I})$$

Judgements, written as $\vdash P :: \mathcal{G}$, states that the process P uses its free names in accordance with $\mathcal{G}$ and can be derived using the inference rules displayed in Figure 1.

Typing Derivations $(\mathcal{D}, \mathcal{E}, \mathcal{F}, \dots)$ are written as $\vdash P :: \mathcal{G}$, indicating that $\mathcal{D}$ is a derivation for the judgement.

Multiplicative Fragment

$$\begin{array}{c} \frac{\vdash P :: \mathcal{G} | \Gamma, x : A | \Delta, y : A^\perp}{\vdash \nu xy P :: \mathcal{G} | \Gamma, \Delta} \text{CUT} \quad \frac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P | Q :: \mathcal{G} | \mathcal{H}} \text{MIX} \quad \frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0 \\ \frac{\vdash P :: \Gamma, y : A | \Delta, x : B}{\vdash x[y].P :: \Gamma, \Delta, x : A \otimes B} \otimes \quad \frac{\vdash P :: \emptyset}{\vdash x[] . P :: x : \mathbf{1}} \mathbf{1} \quad \frac{\vdash P :: \Gamma, y : A, x : B}{\vdash x(y).P :: \Gamma, x : A \wp B} \wp \quad \frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x : \perp} \perp \end{array}$$

Additional rules for Additives, Quantifiers and Exponentials

$$\begin{array}{c} \frac{\vdash P :: \Gamma, x : A}{\vdash x[L].P :: \Gamma, x : A \oplus B} \oplus_1 \quad \frac{\vdash P :: \Gamma, x : B}{\vdash x[R].P :: \Gamma, x : A \oplus B} \oplus_2 \quad \frac{\vdash P :: \Gamma, x : A \quad \vdash Q :: \Gamma, x : B}{\vdash x.\text{case}\{L:P, R:Q\} :: \Gamma, x : A \& B} \& \\ \frac{}{\vdash x \leftrightarrow y :: x : A^\perp, y : A} \text{AX} \quad \frac{\vdash P :: \Gamma, x : A}{\vdash x[\text{USE}].P :: \Gamma, x : ?A} ? \quad \frac{\vdash P :: \Gamma}{\vdash x[\text{DISP}].P :: \Gamma, x : ?A} \text{w} \quad \frac{\vdash P :: \Gamma, x : ?A, x' : ?A}{\vdash x[\text{DUP}](x').P :: \Gamma, x : ?A} \text{c} \\ \frac{\vdash P :: z_1 : ?B_1, \dots, z_n : ?B_n, x : A}{\vdash !x\langle z_1, \dots, z_n \rangle . \{P\} :: z_1 : ?B_1, \dots, z_n : ?B_n, x : !A} ! \quad \frac{\vdash P :: \Gamma, x : B\{A/X\}}{\vdash x[A].P :: \Gamma, x : \exists X.B} \exists \quad \frac{\vdash P :: \Gamma, x : B \quad X \notin \text{ft}(\Gamma)}{\vdash x(X).P :: \Gamma, x : \forall X.B} \forall \end{array}$$

Fig. 1. π LL, typing rules, adapted from Figure 2 and Figure 6 of Montesi and Peressotti [2021].

The multiplicative rules form the core of the system. **Rule CUT** composes two processes in P sharing dual endpoints $x : A$ and $y : A^\perp$, hiding them via restriction. This is the logical equivalent of communication. **Rule MIX** and **rule MIX₀** compose independent processes in parallel and introduce the terminated process, respectively.

Rules $\otimes$ and $\wp$ type output and input. Note that **rule $\otimes$** types the sent name y in a *separate* environment from the continuation, reflecting that the two are parallel resources. By contrast, **rule $\wp$** keeps y and the continuation x in the *same* environment, reflecting their sequential dependency.

The unit **rules** 1 and $\perp$ type empty output and empty input, where **rule** 1 requires the continuation to be well-typed in the empty hyperenvironment, meaning P is necessarily semantically equivalent to 0 .

The additive rules ($\oplus_1$, $\oplus_2$ and $\&$) type selection and offering of alternative processing paths. Note that **rule** $\&$ requires both branches to be typed in the *same* context Γ , reflecting that the choice of branch is made by the other endpoint.

The exponential rules type server replication. The **rule** $!$ requires all free names except x to be typed using $?$, enforcing that a server only depends on other servers. The **rules** $?$, **w** and **c** allow a client to use, discard, or duplicate a server respectively.

The **rule** **ax** types a link between two dual endpoints, $x \leftrightarrow y$, forwarding all messages sent on x safely to y and vice versa. **Rule** $\exists$ and **rule** $\forall$ type polymorphic communication, where the side condition $X \notin \text{ft}(\Gamma)$ in **rule** $\forall$, ensures the introduced type variable, X , does not accidentally shadow one already existing in Γ .

Example 1.2. The process from **Example 1.1** has the typing $\vdash \mathbf{v}xz(z().y[].0 \mid x[].0) :: y:1$, as shown by the derivation below:

$$\frac{\frac{\frac{\vdash 0 :: \emptyset}{\vdash x[].0 :: x:1} \text{MIX}_0 \quad \frac{\frac{\vdash 0 :: \emptyset}{\vdash y[].0 :: y:1} \text{MIX}_0 \quad 1}{\vdash z().y[].0 :: y:1, z:\perp} \perp}{\vdash x[].0 \mid z().y[].0 :: x:1 \mid y:1, z:\perp} \text{MIX}}{\vdash \mathbf{v}xz(x[].0 \mid z().y[].0) :: y:1} \text{CUT}$$

Note that, for the typing environment $x:1 \mid y:1, z:\perp$ to align with the premise of **rule** **CUT**, we utilise the monoidal properties of environments and hyperenvironments. In particular, using their identity laws, we instantiate the meta-variables of the rule as $\mathcal{G} = \emptyset$ and $\Gamma = \bullet$. Under these instantiations, the premise of **rule** **CUT**: $\vdash P :: \mathcal{G} \mid \Gamma, x:A \mid \Delta, y:A^\perp$ can be viewed as $x:A \mid \Delta, y:A^\perp$. This effectively instantiates the **CUT** over the dual types $A = 1$ and $A^\perp = \perp$ on channels x and z , with $\Gamma = y:1$, while ensuring the restricted endpoints implement dual session behaviours.

1.6 Operational Semantics

The operational semantics of π LL are given as a labelled transition system (LTS) defined over *typing derivations*, in which processes evolve by performing actions on their free names. The semantics follow a Structural Operational Semantics (SOS) style presentation and form the basis for the metatheoretic results of the calculus. We will be focusing on the multiplicative fragment for this part, which is given in **Figure 2**. The full LTS is presented in **Figure 2** and **Appendix B.2** of **Montesi and Peressotti [2021]**.

The operational semantics rely explicitly on the structural equivalence (**rule** $=_\alpha$) to rename restricted variables via α -equivalence prior to a transition. In paper-based presentations, α -equivalence is often handled implicitly through Barendregt's Variable Convention. In a mechanised setting, however, reasoning over named variables requires freshness conditions, capture-avoiding substitution, and explicit renaming lemmas, introducing proof overhead [**Aydemir et al. 2005**; **Charguéraud 2012**]. As discussed in ??, this motivates the use of a different binding representation for the syntax of π LL, one designed to reduce explicit α -equivalence reasoning and simplify the treatment of bound variables.

Actions are defined in the set $Act = \{x[], x(), x[y], x(y) \mid x, y \text{ names}\}$, representing empty output, empty input, name output, and name input respectively. These come together with the silent label τ for internal communication and the parallel composition of labels $l \mid l'$ to form the set of transition labels, defined as $Lbl = \{\tau, l, (l \mid l') \mid l, l' \in Act, i(l) \cap i(l') = \emptyset\}$. The restriction

$$\begin{array}{c}
\frac{\frac{\mathcal{D}}{\vdash P :: \emptyset} \quad 1}{\vdash x[] . P :: x : 1} \xrightarrow{x[]} \frac{\mathcal{D}}{\vdash P :: \emptyset} \quad \frac{\frac{\mathcal{D}}{\vdash P :: \Gamma, x' : A \mid \Delta, x : B} \quad \otimes}{\vdash x[x'] . P :: \Gamma, \Delta, x : A \otimes B} \xrightarrow{x[x']} \frac{\mathcal{D}}{\vdash P :: \Gamma, x' : A \mid \Delta, x : B} \\
\\
\frac{\frac{\mathcal{D}}{\vdash P :: \Gamma} \quad \perp}{\vdash x() . P :: \Gamma, x : \perp} \xrightarrow{x()} \frac{\mathcal{D}}{\vdash P :: \Gamma} \quad \frac{\frac{\mathcal{D}}{\vdash P :: \Gamma, x' : A, x : B} \quad \wp}{\vdash x(x') . P :: \Gamma, x : A \wp B} \xrightarrow{x(x')} \frac{\mathcal{D}}{\vdash P :: \Gamma, x' : A, x : B} \\
\\
\frac{\mathcal{D}}{\vdash P :: \mathcal{G}} \xrightarrow{l} \frac{\mathcal{D}'}{\vdash P' :: \mathcal{G}'} \quad i(l) \cap f(Q) = \emptyset \\
\\
\frac{\frac{\mathcal{D}}{\vdash P :: \mathcal{G}} \quad \frac{\mathcal{E}}{\vdash Q :: \mathcal{H}} \quad \text{MIX}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}} \xrightarrow{l} \frac{\frac{\mathcal{D}'}{\vdash P' :: \mathcal{G}'} \quad \frac{\mathcal{E}}{\vdash Q :: \mathcal{H}} \quad \text{MIX}}{\vdash P' \mid Q :: \mathcal{G}' \mid \mathcal{H}} \quad \text{PAR}_1 \\
\\
\frac{\mathcal{E}}{\vdash Q :: \mathcal{H}} \xrightarrow{l} \frac{\mathcal{E}'}{\vdash Q' :: \mathcal{H}'} \quad i(l) \cap f(P) = \emptyset \\
\\
\frac{\frac{\mathcal{D}}{\vdash P :: \mathcal{G}} \quad \frac{\mathcal{E}}{\vdash Q :: \mathcal{H}} \quad \text{MIX}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}} \xrightarrow{l} \frac{\frac{\mathcal{D}}{\vdash P :: \mathcal{G}} \quad \frac{\mathcal{E}'}{\vdash Q' :: \mathcal{H}'} \quad \text{MIX}}{\vdash P \mid Q' :: \mathcal{G} \mid \mathcal{H}'} \quad \text{PAR}_2 \\
\\
\frac{\frac{\mathcal{D}}{\vdash P :: \mathcal{G}} \xrightarrow{l} \frac{\mathcal{D}'}{\vdash P' :: \mathcal{G}'} \quad \frac{\mathcal{E}}{\vdash Q :: \mathcal{H}} \xrightarrow{l'} \frac{\mathcal{E}'}{\vdash Q' :: \mathcal{H}'} \quad i(l \mid l') \cap f(P \mid Q) = \emptyset}{\frac{\frac{\mathcal{D}}{\vdash P :: \mathcal{G}} \quad \frac{\mathcal{E}}{\vdash Q :: \mathcal{H}} \quad \text{MIX}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}} \xrightarrow{l \mid l'} \frac{\frac{\mathcal{D}'}{\vdash P' :: \mathcal{G}'} \quad \frac{\mathcal{E}'}{\vdash Q' :: \mathcal{H}'} \quad \text{MIX}}{\vdash P' \mid Q' :: \mathcal{G}' \mid \mathcal{H}'}} \quad \text{SYN} \\
\\
\frac{P =_{\alpha} Q \quad \frac{\mathcal{E}}{\vdash Q :: \mathcal{G}} \xrightarrow{l} \frac{\mathcal{E}'}{\vdash Q' :: \mathcal{G}'}}{\frac{\mathcal{D}}{\vdash P :: \mathcal{G}} \xrightarrow{l} \frac{\mathcal{E}'}{\vdash Q' :: \mathcal{G}'}} =_{\alpha} \frac{\frac{\mathcal{D}}{\vdash P :: \mathcal{G} \mid x : 1 \mid \Gamma, y : \perp} \quad \frac{\mathcal{D}'}{\vdash P' :: \mathcal{G} \mid \Gamma}}{\frac{\mathcal{D}}{\vdash P :: \mathcal{G} \mid x : 1 \mid \Gamma, y : \perp} \quad \frac{\mathcal{D}'}{\vdash P' :: \mathcal{G} \mid \Gamma}} \quad 1\perp \\
\\
\frac{\frac{\mathcal{D}}{\vdash P :: \mathcal{G} \mid x : 1 \mid \Gamma, y : \perp} \quad \text{CUT}}{\vdash vxy P :: \mathcal{G} \mid \Gamma} \xrightarrow{\tau} \frac{\frac{\mathcal{D}'}{\vdash P' :: \mathcal{G} \mid \Gamma, x : B \mid \Delta, x' : A \mid \Xi, y : B^{\perp}, y' : A^{\perp}} \quad \text{CUT}}{\vdash vx'y' P' :: \mathcal{G} \mid \Gamma, x : B \mid \Delta, \Xi, y : B^{\perp}} \quad \otimes \otimes \\
\\
\frac{\frac{\mathcal{D}}{\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}} \quad \text{CUT}}{\vdash vxy P :: \mathcal{G} \mid \Gamma, \Delta} \xrightarrow{l} \frac{\frac{\mathcal{D}'}{\vdash P' :: \mathcal{G}' \mid \Gamma', x : A \mid \Delta', y : A^{\perp}} \quad \text{CUT}}{\vdash vx'y' P' :: \mathcal{G}' \mid \Gamma', \Delta'} \quad \text{RES} \\
\\
\frac{\frac{\mathcal{D}}{\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^{\perp}} \quad \text{CUT}}{\vdash vxy P :: \mathcal{G} \mid \Gamma, \Delta} \xrightarrow{l} \frac{\frac{\mathcal{D}'}{\vdash P' :: \mathcal{G}' \mid \Gamma', x : A \mid \Delta', y : A^{\perp}} \quad \text{CUT}}{\vdash vx'y' P' :: \mathcal{G}' \mid \Gamma', \Delta'} \quad \text{RES}
\end{array}$$

Fig. 2. π MLL, transition rules for derivations, as presented in Figure 2 of Montesi and Peressotti [2021].

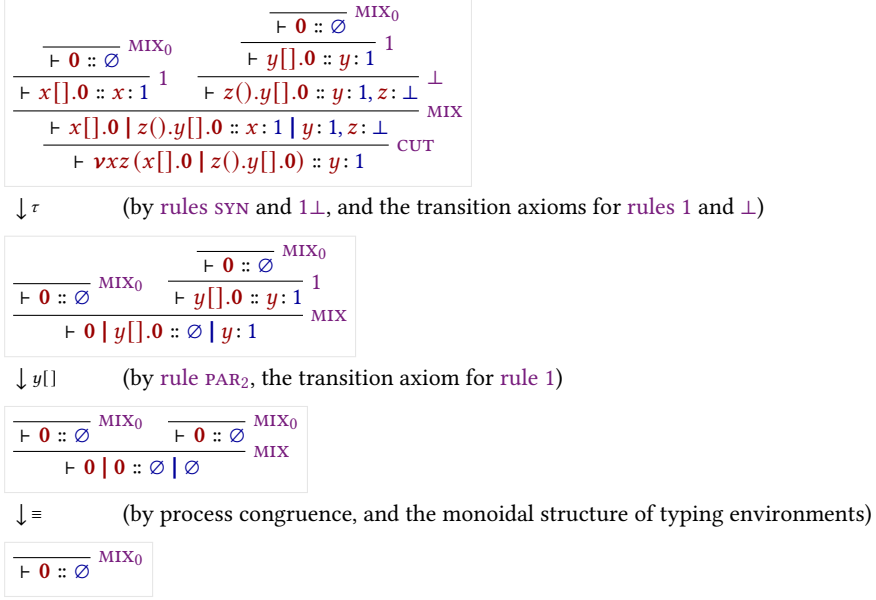


Fig. 3. An execution for the derivation in Example 1.2.

imposed on the parallel composition of labels exists to ensure two labels do not introduce the same name, which would break linear resource management.

The free and introduced names of labels are defined by $f(x[]) = f(x()) = f(x[y]) = f(x(y)) = \{x\}$, $f(\tau) = i(\tau) = \emptyset$, and $f(l \mid l') = f(l) \cup f(l')$, and introduced names by $i(x[]) = i(x()) = \emptyset$, $i(x[y]) = i(x(y)) = \{y\}$, and $i(l \mid l') = i(l) \cup i(l')$. The parallel label $l \mid l'$ additionally requires $i(l) \cap i(l') = \emptyset$, ensuring no two components introduce the same name.

Example 1.3. Figure 3 shows a possible execution for the derivation of the process from Example 1.1.

Each logical rule (1 , $\perp$, $\otimes$, $\wp$) has a corresponding transition axiom. These follow the prefix-continuation pattern $\pi.P \xrightarrow{\pi} P$, where performing the action π leads to the remainder of the process. Rule SYN allows for the pairing of concurrent actions via $l \mid l'$, which is used to facilitate communication by letting two endpoints connected by a restriction perform compatible actions. When two compatible actions are performed over endpoints connected by a restriction, rules $1\perp$ and $\otimes\wp$ simplify the term into an internal τ -transition, modeling the synchronisation of the session.

1.7 Process Semantics and Erasure

While the typing derivations of π LL dictate the valid interactions within a session, the underlying process terms possess a standard, untyped operational behavior that closely mirrors the classic π -calculus. To formally relate the typed derivations to their untyped execution, we follow Montesi and Peressotti [2021] and define an erasure function, $\text{proc} : \text{Der} \rightarrow \text{Proc}$, which strips away the typing information from a derivation. Recursively, proc maps the head of a typing derivation to the specific process constructor it introduces, continuing structurally onto the process's continuation. For any well-typed π LL term, $\text{proc}(\mathcal{D})$ effectively extracts the pure process term from the derivation's conclusion.

$$\begin{array}{c}
\frac{\frac{\frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0}{\vdash v[].\mathbf{0} :: v:1} 1}{\vdash w().v[].\mathbf{0} :: v:1, w:\perp} \perp \quad \frac{\frac{\frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0}{\vdash x[].\mathbf{0} :: x:1} 1}{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x:1 \mid y:1, z:\perp} \perp \\
\frac{\frac{\frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0}{\vdash y[].\mathbf{0} :: y:1} 1}{\vdash z().y[].\mathbf{0} :: y:1, z:\perp} \perp \\
\frac{\frac{\vdash w().v[].\mathbf{0} :: v:1, w:\perp}{\vdash (w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: v:1, w:\perp \mid x:1 \mid y:1, z:\perp} \text{MIX}}{\vdash vxz(w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: v:1, w:\perp \mid y:1} \text{CUT}
\end{array}$$

Fig. 4. A derivation for the process $vxz(w().y[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0})$

Example 1.4. Consider the well-typed process P formed by adding a third parallel component to the derivation of [Example 1.2](#) before the cut (A derivation of which can be seen on [Figure 4](#)):

$$P = vxz(w().v[].\mathbf{0} \mid x[].\mathbf{0} \mid z().y[].\mathbf{0})$$

The component containing w and v is independent from the component containing x , z , and y . This allows their respective actions to interleave freely, resulting in the following execution traces, illustrating the concurrency of the calculus:

$$\begin{array}{ccc}
P \xrightarrow{\tau} \xrightarrow{w()} \xrightarrow{v[]} \xrightarrow{y[]} \mathbf{0} & P \xrightarrow{\tau} \xrightarrow{w()} \xrightarrow{y[]} \xrightarrow{v[]} \mathbf{0} & P \xrightarrow{\tau} \xrightarrow{y[]} \xrightarrow{w()} \xrightarrow{v[]} \mathbf{0} \\
P \xrightarrow{w()} \xrightarrow{v[]} \xrightarrow{\tau} \xrightarrow{y[]} \mathbf{0} & P \xrightarrow{w()} \xrightarrow{\tau} \xrightarrow{v[]} \xrightarrow{y[]} \mathbf{0} & P \xrightarrow{w()} \xrightarrow{\tau} \xrightarrow{y[]} \xrightarrow{v[]} \mathbf{0}
\end{array}$$

The semantics for process terms must inherently agree with the semantics of derivations for well-typed terms. As formulated by [Montesi and Peressotti \[2021\]](#), this relationship is established functorially. Let $\mathcal{P}(X) = \{X' \mid X' \subseteq X\}$ and $\mathcal{L}(X) = X \times \text{Lbl}$ be endofunctors representing the states and labeled transitions, respectively. The erasure function *proc* acts as a homomorphism from LTS of derivations (lts_d) to the LTS of processes (lts_p), ensuring that the square of these LTS mappings commutes.

In the context of operational semantics, this functorial homomorphism yields an Erasure theorem. It guarantees that the Labeled Transition System for derivations and the SOS for untyped processes simulate each other exactly:

THEOREM 1.5 (ERASURE). *The process LTS (lts_p) enjoys erasure with respect to the derivation LTS (lts_d). Concretely, for any valid typing derivation $\mathcal{D}$ and label $l \in \text{Lbl}$:*

- (1) **Soundness of Erasure:** *If $\mathcal{D} \xrightarrow{l} \mathcal{D}'$, then $\text{proc}(\mathcal{D}) \xrightarrow{l} \text{proc}(\mathcal{D}')$.*
- (2) **Completeness of Erasure:** *If $\text{proc}(\mathcal{D}) \xrightarrow{l} P'$, then there exists a derivation $\mathcal{D}'$ such that $\mathcal{D} \xrightarrow{l} \mathcal{D}'$ and $\text{proc}(\mathcal{D}') = P'$.*

COROLLARY 1.6 (TYPABILITY PRESERVATION). *If P is well-typed and $P \xrightarrow{l} P'$, then P' is well-typed.*

This theorem allows us to define the operational target for πLL processes without the syntactic overhead of typing environments. Using the erasure formulation, we can transform the operational semantics for derivations directly into an SOS specification for processes.

The process LTS must state freshness and name-disjointness conditions explicitly because it describes process evolution independently of typing information. For example, synchronising actions must not introduce the same fresh name, as expressed by conditions such as $i(l) \cap i(l') = \emptyset$. For processes arising from valid typing derivations, these obligations are supported by the linear resource structure of πLL : typing restricts the combination of hyperenvironment components to

$$\begin{array}{l}
\text{491} \quad x[x'].P \xrightarrow{x[x']} P \quad x(x').P \xrightarrow{x(x')} P \quad x[].P \xrightarrow{x[]} P \quad x().P \xrightarrow{x()} P \quad i(l|l') = i(l) \cup i(l') \\
\text{492} \quad \frac{P \xrightarrow{l} P' \quad i(l) \cap f(Q) = \emptyset}{P | Q \xrightarrow{l} P' | Q} \text{PAR}_1 \quad \frac{Q \xrightarrow{l} Q' \quad i(l) \cap f(P) = \emptyset}{P | Q \xrightarrow{l} P | Q'} \text{PAR}_2 \quad f(l|l') = f(l) \cup f(l') \\
\text{493} \quad \frac{P \xrightarrow{l} P' \quad Q \xrightarrow{l'} Q' \quad i(l|l') \cap f(P | Q) = \emptyset}{P | Q \xrightarrow{l|l'} P' | Q'} \text{SYN} \quad \frac{P \xrightarrow{x[x']|y(y')} P' \quad \text{v}xy P \xrightarrow{\tau} \text{v}xy \text{v}x'y' P'}{\text{v}xy P \xrightarrow{\tau} \text{v}xy P'} \otimes \otimes \quad i(\tau) = \emptyset \\
\text{494} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad f(\tau) = \emptyset \\
\text{495} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad i(x[]) = i(x()) = \emptyset \\
\text{496} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad f(x[]) = f(x()) = \{x\} \\
\text{497} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad i(x[x']) = i(x(x')) = \{x'\} \\
\text{498} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad f(x[x']) = f(x(x')) = \{x'\} \\
\text{499} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad f(x[x']) = f(x(x')) = \{x'\} \\
\text{500} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad f(x[x']) = f(x(x')) = \{x'\} \\
\text{501} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad f(x[x']) = f(x(x')) = \{x'\} \\
\text{502} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad f(x[x']) = f(x(x')) = \{x'\} \\
\text{503} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad f(x[x']) = f(x(x')) = \{x'\} \\
\text{504} \quad \frac{P \xrightarrow{x[]|y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\perp \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha} \quad f(x[x']) = f(x(x')) = \{x'\}
\end{array}$$

Fig. 5. π MLL, transition rules for processes.

disjoint domains and accounts for the names used by the process. Thus the process semantics records the operational side conditions explicitly, while the typed semantics explains why they are satisfied by well-typed execution. The corresponding evolution of these typing resources is described by the environment semantics introduced next.

1.8 Semantics of Typing Environments

Just as processes evolve through computation, the typing environments that govern them must also evolve to accurately track the state of a session. In π LL, types capture communication protocols. Consequently, the transition rules for typing environments specify exactly which actions are permitted and how the available resources are consumed by those actions.

Similar to how we defined the process erasure function, we again follow [Montesi and Peressotti \[2021\]](#) and define a function $env : Der \rightarrow Env$ that maps a valid typing derivation directly to the hyperenvironment in its conclusion (i.e., $env(\vdash P :: \mathcal{G}) = \mathcal{G}$). Using this extraction, [Montesi and Peressotti \[2021\]](#) derive a LTS specifically for typing environments (lts_e), shown in [Figure 6](#).

$$\begin{array}{l}
\text{511} \quad x : 1 \xrightarrow{x[]} \emptyset \quad \Gamma, x : \perp \xrightarrow{x()} \Gamma \quad \Gamma, \Delta, x : A \otimes B \xrightarrow{x[x']} \Gamma, x' : A | \Delta, x : B \quad \Gamma, x : A \wp B \xrightarrow{x(x')} \Gamma, x' : A, x : B \\
\text{512} \quad \frac{\mathcal{G} \xrightarrow{l} \mathcal{G}'}{\mathcal{G} | \mathcal{H} \xrightarrow{l} \mathcal{G}' | \mathcal{H}} \text{PAR}_1 \quad \frac{\mathcal{H} \xrightarrow{l} \mathcal{H}'}{\mathcal{G} | \mathcal{H} \xrightarrow{l} \mathcal{G} | \mathcal{H}'} \text{PAR}_2 \quad \frac{\mathcal{G} \xrightarrow{l} \mathcal{G}' \quad \mathcal{H} \xrightarrow{l'} \mathcal{H}'}{\mathcal{G} | \mathcal{H} \xrightarrow{l|l'} \mathcal{G}' | \mathcal{H}'} \text{SYN} \\
\text{513} \quad \frac{\mathcal{G} | \Gamma, x : A^{\perp} | \Delta, y : A \xrightarrow{l} \mathcal{G}' | \Gamma', x : A^{\perp} | \Delta', y : A}{\mathcal{G} | \Gamma, \Delta \xrightarrow{l} \mathcal{G}' | \Gamma', \Delta'} \text{RES} \quad \frac{\mathcal{G} | x : 1 | \Gamma, y : \perp \xrightarrow{x[]|y()} \mathcal{G} | \Gamma}{\mathcal{G} | \Gamma \xrightarrow{\tau} \mathcal{G} | \Gamma} 1\perp \\
\text{514} \quad \frac{\mathcal{G} | \Gamma, \Delta, x : A \otimes B | \Xi, y : A^{\perp} \wp B^{\perp} \xrightarrow{x[x']|y(y')} \mathcal{G} | \Gamma, x : B | \Delta, x' : A | \Xi, y : B^{\perp}, y' : A^{\perp}}{\mathcal{G} | \Gamma, \Delta, \Xi \xrightarrow{\tau} \mathcal{G} | \Gamma, \Delta, \Xi} \otimes \otimes \\
\text{515} \quad \frac{\mathcal{G} | \Gamma, \Delta, x : A \otimes B | \Xi, y : A^{\perp} \wp B^{\perp} \xrightarrow{x[x']|y(y')} \mathcal{G} | \Gamma, x : B | \Delta, x' : A | \Xi, y : B^{\perp}, y' : A^{\perp}}{\mathcal{G} | \Gamma, \Delta, \Xi \xrightarrow{\tau} \mathcal{G} | \Gamma, \Delta, \Xi} \otimes \otimes
\end{array}$$

Fig. 6. π MLL, transition rules for typing environments.

In this LTS, observable actions actively consume resources from the environment. For example, if a hyperenvironment contains a waiting channel $x : \perp$, performing an input action $x()$ will consume this type, removing it from the resulting environment. Conversely, internal communications do not alter the available external resources. This is formalised by the τ -reflexivity axiom, $\mathcal{G} | \Gamma \xrightarrow{\tau} \mathcal{G} | \Gamma$, which states that any valid hyperenvironment transitions to itself under a silent τ -action.

The rules in Figure 6 define which transitions are available at the environment level. For a well-typed process, however, process behaviour and resource evolution must step in unison such that whenever the process performs an action, its typing environment must be able to perform the corresponding transition. This correspondence is the property of **session fidelity**. Just as the *proc* projection extracts the process behaviour of a typed transition, the *env* projection extracts its corresponding environment evolution.

THEOREM 1.7 (SESSION FIDELITY). *Let $\mathcal{D}$ be a valid typing derivation such that $\text{env}(\mathcal{D}) = \mathcal{G}$ and $\text{proc}(\mathcal{D}) = P$. If the process transitions such that $P \xrightarrow{l} P'$, then the environment can mimic this transition $\mathcal{G} \xrightarrow{l} \mathcal{G}'$, and there exists a new valid derivation $\mathcal{D}'$ for P' under $\mathcal{G}'$.*

To illustrate Session Fidelity in action, we can observe how the environment is constrained to follow the exact execution traces of the process it types.

Example 1.8 (Session Fidelity in Execution). Recall the process derivation from Example 1.4, initially typed by the hyperenvironment $\mathcal{G} = v : 1, w : \perp \mid y : 1$. The process first performs an observable input action on w . By session fidelity, this process transition is mirrored by the corresponding environment transition, consuming the resource typed by $\perp$:

$$\begin{aligned} v : 1, w : \perp \mid y : 1 &\xrightarrow{w()} v : 1 \mid y : 1 \\ &\xrightarrow{\tau} v : 1 \mid y : 1 \\ &\xrightarrow{y[]} v : 1. \end{aligned}$$

After the first transition, the process derivative is typed by $v : 1 \mid y : 1$. Viewed in isolation, the environment transition system admits an output action on y , since $y : 1$ is available as a resource. However, session fidelity does not state that every environment transition can be realised by the current process. Rather it states that each process transition has a corresponding environment transition. In the process term, the action $y[]$ remains guarded until an internal synchronisation has taken place. The environment therefore mirrors the intermediate τ -transition without changing its resources, and only afterwards mirrors the observable output on y by consuming the final $y : 1$ component.

Session fidelity is a central correctness property of a session-typed calculus: it expresses that process evolution is matched by the evolution of its typing resources. In the paper presentation, this correspondence is supported by implicit conventions for fresh names, bound variables, and structural rearrangement. In a proof assistant, these conventions must instead be represented explicitly and manipulated by proved lemmas. Establishing that types are consumed correctly therefore requires a concrete treatment of binding, opening, substitution, and freshness. The next chapter develops this binding architecture, which provides the foundation for the subsequent Lean formalisation and its metatheory.

1.9 Managing Bound Variables

Having established the operational semantics of πLL and the ultimate goal of verifying Session Fidelity, we must address the treatment of bound variables and freshness, a central technical challenge that underpins the entire formalisation.

As we have seen, many constructs in πLL introduce scoped identifiers. Restriction binds pairs of session endpoints, input constructs bind received channel names, polymorphic communication binds type variables, and duplication constructs introduce fresh channel endpoints. These bindings appear throughout both the process syntax and the labelled transition system.

The concrete choice of bound and introduced names is semantically irrelevant provided the binding structure is preserved. For example, renaming a restricted channel does not change the behaviour of a process. However, reasoning formally about substitutions, transitions, and typing derivations requires these names, scopes, and freshness conditions to be tracked explicitly.

This principle is formalised as α -equivalence, which identifies terms that differ only in the names of their bound variables. For instance, the following two processes are α -equivalent:

$$\mathbf{vab}(a[\cdot].0 \mid b().x[\cdot].0) =_{\alpha} \mathbf{vzw}(z[\cdot].0 \mid w().x[\cdot].0)$$

The operational semantics of π LL relies on α -equivalence using **rule** $=_{\alpha}$ to rename bound variables on the fly, ensuring that restricted scopes and input prefixes do not accidentally capture free names as processes evolve.

Substitution raises this exact issue. Consider the substitution where the free name x is replaced by z in the right-hand process from the previous example:

$$\mathbf{vzw}(z[\cdot].0 \mid w().x[\cdot].0)$$

Here z is already bound by the restriction, so naively replacing x with z would cause the restriction to capture the newly inserted z . Before the substitution can proceed, and because restriction binds session endpoints as pairs, both the bound z and w must be α -renamed to fresh names, such as to a and b . Similarly, any binder within the process that would capture the substituted name must also be renamed before substitution continues.

In paper-based presentations this is commonly handled by *Barendregt's variable convention*, which assumes bound variables are always chosen distinct from free variables and one another whenever convenient. This makes proofs readable but remains an informal meta-level assumption.

A proof assistant requires every freshness condition and renaming step to be represented and verified explicitly [Aydemir et al. 2005]. A direct mechanisation using named variables therefore introduces substantial proof overhead for capture avoidance and α -equivalence reasoning. To avoid this, the formalisation adopts a *locally nameless* representation of binding, which eliminates α -equivalence as a proof obligation by construction. This is the subject of the following chapter.

References

- Brian E. Aydemir, Aaron Bohannon, Matthew Fairbairn, J. Nathan Foster, Benjamin C. Pierce, Peter Sewell, Dimitrios Vytiniotis, Geoffrey Washburn, Stephanie Weirich, and Steve Zdancewic. 2005. Mechanized Metatheory for the Masses: The PoplMark Challenge. In *Theorem Proving in Higher Order Logics*, Joe Hurd and Tom Melham (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 50–65.
- Luis Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR*. 222–236.
- Marco Carbone, Sam Lindley, Fabrizio Montesi, Carsten Schürmann, and Philip Wadler. 2016. Coherence Generalises Duality: A Logical Explanation of Multiparty Session Types. In *27th International Conference on Concurrency Theory, CONCUR 2016, August 23–26, 2016, Québec City, Canada (LIPIcs, Vol. 59)*, Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 33:1–33:15. doi:10.4230/LIPIcs.CONCUR.2016.33
- Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (2012), 363–408. doi:10.1007/s10817-011-9225-2
- Roberto Di Cosmo and Dale Miller. 2023. Linear Logic. In *The Stanford Encyclopedia of Philosophy* (fall 2023 ed.), Edward N. Zalta and Uri Nodelman (Eds.). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/fall2023/entries/logic-linear/>
- Jean-Yves Girard. 1987. Linear Logic. *Theor. Comput. Sci.* 50 (1987), 1–102. doi:10.1016/0304-3975(87)90045-4
- Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, I. *Inf. Comput.* 100, 1 (1992), 1–40.
- Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the π -calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818
- Philip Wadler. 2014. Propositions as sessions. *Journal of Functional Programming* 24, 2–3 (2014), 384–418. Also: ICFP, pages 273–286, 2012.